Morph Channel

A Cell-Native Protocol Construction for Sponsored State Evidence and Local Channel Factories on CKB

Arthur Zhang

May 2026

Abstract

CKB's Cell model allows a channel to separate the object that holds value from the object that carries dispute evidence. Morph Channel uses this separation to keep the funding identity stable whilst unilateral disputes advance a State Cell containing signed state evidence. Publication transactions are met from sponsor-owned Cells, so channel reserve, business CKB, and xUDT balances remain outside the fee ledger.

The construction defines a State Header signing domain that binds chain context, channel identity, funding epoch, funding anchor, vault-set commitment, state number, asset registry, settlement descriptor, and challenge policy, whilst leaving sponsor coin selection unsigned. State Cell, vault-lock, and sponsor-budget validation are specified as script-level predicates. Partition conservation checks occupied capacity, channel reserve, business CKB, and each xUDT type separately, with transaction fees charged only to the sponsor partition.

For bilateral channels, Morph begins with plaintext witnesses so force-close transactions remain auditable. For factories, authority is commitment-based: access manifests, local proofs, touched leaves, and an outer envelope define the admissible transition before proof bytes are interpreted. Under signature unforgeability, hash collision resistance, CKB single-spend enforcement, and a challenge window that covers detection, publication, confirmation, fee bumping, and reorganisation margin, newer valid state evidence supersedes older settling evidence before finalisation. The construction is limited to current CKB primitives: Cells, lock scripts, type scripts, witnesses, `cell_deps`, and `since`.

Keywords: CKB, Cell model, payment channels, channel factories, sponsored fees, partition conservation, xUDT, state evidence

Contents

1 Introduction and Motivation

Payment channels and state channels keep frequent updates off-chain whilst preserving unilateral settlement on-chain when cooperation fails. The Lightning Network [1] is the canonical payment-channel construction, whilst eltoo [2] isolates a replacement-based rule: during a dispute, newer mutually signed state should defeat stale evidence. Channel factories [3] extend this idea to shared funding structures that support many off-chain relationships.

CKB exposes a different design point. A Cell has data, a lock script, an optional type script, capacity, and a stable outpoint [6, 7]. A CKB channel can therefore make scripts recognise two different objects: a stable value anchor and a moving state-evidence Cell. This gives the design question:

> What protocol structure is required so that CKB can express a channel whose funding object remains stable, while disputes move only signed state evidence?

Morph Channel keeps channel value anchored in stable Cells, advances a distinct State Cell during disputes, and meets publication fees from sponsor-owned funds rather than channel-owned value. It uses the newer-state-wins principle, but implements it through CKB scripts, State Cells, relative `since`, and explicit value partitioning rather than Bitcoin-style output reshaping.

1.1 Scope

The paper focuses on script-level channel and factory architecture. It does not attempt to solve routing, liquidity discovery, gossip, privacy networks, watchtower markets, or factory governance. It also avoids any assumption of sub-second finality, DAG ordering, native channel transaction classes, or CKB node modification. All claims are bounded by existing CKB primitives.

The scope is the construction itself: threat model, object authority, transaction invariants, deployability, and benchmark obligations. The paper does not claim readiness for production deployment.

1.2 Design Thesis

The central thesis is:

> *A CKB channel becomes simpler when funding identity, state authority, and publication fee responsibility are separate protocol objects.*

Morph treats the funding object, the signed state, and the fee-paying transaction inputs as separate authorities. The funding object identifies channel-owned value. The State Cell and participant signatures determine how that value may settle. Sponsor inputs fund publication without changing channel balances. This separation keeps unilateral exit independent of wallet coin selection and makes multi-asset conservation auditable.

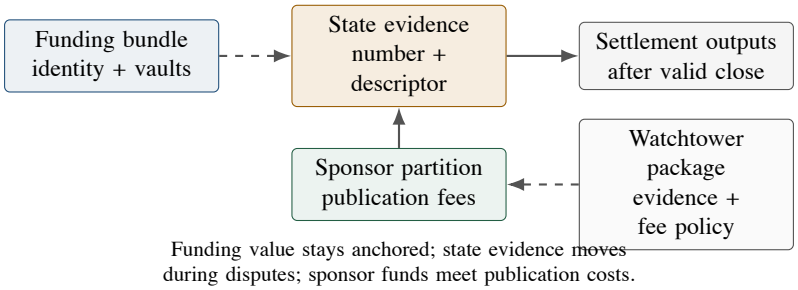


Figure 1: High-level decomposition. The channel value object, the moving dispute evidence, and the fee-paying sponsor funds are separate protocol objects.

2 Problem Statement and Current Snapshot

2.1 The Funding-State Coupling Problem

In many UTXO-oriented channel designs, settlement value and channel state are operationally coupled: a state update is represented by a new arrangement of future spend paths over the funding output. This coupling is natural in Bitcoin-like systems, but it is not inevitable on CKB. Since Cells can carry explicit typed state, a channel may maintain a stable funding anchor and move a separate state object.

If the value-holding object is also the state-update object, a force-close template can mix settlement authorisation, fee selection, and asset accounting in one place. Morph separates those checks: the vault protects channel value, the State Cell carries dispute evidence, and sponsor inputs fund the carrier transaction.

2.2 The Fee-Boundary Problem

A fee cap and a zero channel-paid fee policy are not equivalent. A fee cap says that channel value may be consumed up to a limit. A zero channel-paid fee invariant says that publication fees are not part of channel-owned value at all. CKB makes this distinction particularly important because capacity simultaneously represents storage resource, fee medium, and sometimes user-visible business value.

2.3 The Authority Object Problem

A channel protocol should not let scripts accept several competing authority sources. In a poorly separated design, authority may drift among:

(1) the funding Cell;

(2) a plaintext witness;

(3) a local plaintext snapshot;

(4) a root commitment;

(5) a transaction body containing incidental sponsor inputs.

Morph Channel reduces authority to a signed state header plus the admissible witness or proof required by its mode. Transaction bodies remain reconstructible wrappers around that evidence.

3 Research Contributions and Design Principles

3.1 Main Contributions

This paper makes nine contributions.

C1. **Stable funding and moving evidence.** It defines a dual-track Cell construction in which channel value remains in stable funding and vault Cells, while disputes move a State Cell.

C2. **State-header-centric signing.** It specifies a canonical signing domain that binds identity and semantics while excluding sponsor coin-selection details.

C3. **Sponsored publication.** It defines a sponsor-owned publication partition and requires unilateral publication to be fee-bumpable without counterparty re-signing.

C4. **Partition conservation.** It replaces fee-cap reasoning with exact conservation of channel-owned value and fee extraction from sponsor-owned value.

C5. **Splice-aware funding identity.** It distinguishes stable logical channel identity from the current funding epoch, funding anchor, and vault set, so resizing a channel does not make stale evidence reusable.

C6. **Bilateral/factory witness separation.** It argues that bilateral channels should begin with plaintext witnesses, while factories require commitment-authoritative local proofs.

C7. **Envelope-first factory authorisation.** It separates admission of a factory transition from interpretation of its proof body: a bounded outer claim names the transition family and commits to one evidence body.

C8. **Challenge-window budgeting.** It derives the relative-`since` delay as a deployment parameter covering detection, polling, propagation, confirmation, fee volatility, and reorganisation margin.

C9. **Factory non-interference.** It identifies non-interference as the condition required before a factory action can safely reduce its signing set below all participants.

3.2 Design Principles

The construction is organised around five design principles.

P1. **Script-level deployability.** All mechanisms must be expressible by deployable CKB scripts.

P2. **No hidden fee leakage.** Channel-owned capacity must not silently pay publication fees.

P3. **Narrow descriptors.** Settlement descriptors authorise output derivation and should not become an application runtime.

P4. **Envelope before factory body.** A factory proof body is admissible only after a bounded outer claim identifies the transition family and commits to that body.

P5. **Splice freshness.** Watchtower and settlement evidence must be funding-epoch and vault-set aware, so a package from an earlier funding anchor cannot govern a later resized channel.

Lower-level safety obligations are stated as invariants and validation rules in the sections that use them: vault finalisation requires an authentic current State Cell; challenge maturity is a canonical relative-block CKB `since` condition; non-interference does not authorise value-bearing right increases; descriptor evolution is signed state semantics; value-bearing Cells must materialise exactly; state retirement must not orphan value; and unverifiable sponsor deadlines remain outside script-enforced policy.

4 System Model and Threat Assumptions

4.1 Ledger Model

Let $\mathcal{U}$ denote the set of live CKB Cells. A transaction consumes a finite input set $I \subseteq \mathcal{U}$, may reference read-only cells through `cell_deps`, and creates a finite output set O. CKB consensus verifies input availability, script validity, capacity constraints, and `since` maturity.

Assumption 1 (Base-layer correctness)**.** *CKB consensus correctly enforces transaction validity, script execution, occupied-capacity checks, and* `since` *semantics.*

Assumption 2 (Cryptographic soundness)**.** *The signature scheme is existentially unforgeable under chosen-message attacks, and the hash function used for state commitments is collision resistant.*

4.2 Actors

The protocol includes channel participants, optional watchtowers, standard wallets providing sponsor funds, and CKB miners/validators. A watchtower may observe chain events and publish newer state evidence, but it should not hold custody over channel-owned value.

4.3 Adversarial Goals

An adversary may attempt to:

A1. publish a stale state and let it finalise;

A2. replay a state signature across another channel or funding anchor;

A3. reference a descriptor-shaped but non-authoritative funding Cell;

A4. spend vault value without current state authorisation;

A5. make channel-owned capacity pay publication fees;

A6. confuse CKB reserve, CKB business balance, xUDT amounts, and sponsor change;

A7. exploit factory locality to harm untouched participants.

5 Formal Objects

5.1 Channel Configuration

A Morph channel configuration is a tuple

$$\Gamma = (\mathsf{chain}, \mathsf{cid}, \mathsf{participants}, \mathsf{fund}, \mathsf{assets}, \mathsf{descriptor}, \mathsf{challenge}, \mathsf{version}).$$

Here fund identifies the current funding epoch, funding anchor, and vault-set commitment, assets commits to the asset registry, descriptor commits to the current settlement-output derivation, and challenge specifies the relative finalisation policy. The descriptor commitment is state semantics: descriptor instances may advance with newly signed state, while descriptor grammar, version boundaries, and authorisation rules remain protocol semantics.

5.2 Stable Funding Bundle

Definition 1 (Funding Bundle)**.** *The funding bundle $\mathcal{F}$ consists of a channel identity Cell and zero or more vault Cells:*

$$\mathcal{F} = (\mathsf{Fund}, \mathsf{Vault}_1, \ldots, \mathsf{Vault}_k).$$

The bundle carries channel identity and channel-owned value. It is stable during ordinary off-chain state progression. A splice may replace the current funding anchor and vault set, but only by advancing the signed funding epoch while preserving the logical channel identity.

Definition 2 (Vault Authorisation)**.** *A vault Cell is admissible only if its lock or type script rejects every spend that is not authorised by the authentic currently valid State Cell and the committed settlement descriptor. A Cell whose data merely decodes as a state header is not state authority unless its script identity also matches the channel construction.*

Definition 3 (Current State Pointer)**.** *For a channel identifier* cid*, the current state pointer is the unique live State Cell that can be consumed by the next unilateral publication, supersession, or finalisation operation. The protocol does not require a global registry for this pointer; uniqueness follows from creating exactly one initial State Cell and requiring every state transition to consume the current one and create exactly one successor.*

Definition 4 (Initial State Uniqueness)**.** *A valid* FUND *transaction creates exactly one initial State Cell for the channel's funding anchor. The initial State Cell, the Fund Cell, and all vault locks must bind the same immutable channel genesis identity. A deployable instance expresses this binding in State Cell type arguments, using a Type-ID-style uniqueness rule or an equivalent script-level construction available on current CKB.*

This uniqueness condition is a construction requirement, not a global lookup assumption. The scripts do not need to scan the chain for competing state tracks. Instead, they must make every valid vault spend and every valid State Cell transition refer to the same genesis identity, so a descriptor-shaped but non-authoritative State Cell cannot control the vaults.

5.3 Moving State Evidence

Definition 5 (State Evidence)**.** *State evidence is a signed object*

$$E_n = (\mathsf{Header}_n, \mathsf{Payload}_n, \Pi_n, \Sigma_n),$$

where Header_n *is the canonical state header,* $\mathsf{Payload}_n$ *is plaintext or committed state material,* Π_n *is an optional proof package, and* Σ_n *is the participant authorisation set.*

Definition 6 (Canonical State Header)**.** *The canonical state header contains:*

```
StateHeader {
  protocol_version
  chain_id
  signature_scheme_id
  channel_id_or_genesis_identity
  funding_epoch
  funding_anchor_identity
  vault_set_commitment
  state_number
  mode
  phase
  participants_commitment
  asset_registry_commitment
  settlement_descriptor_commitment
  descriptor_version
  payload_commitment
  challenge_policy_commitment
  state_layout_version
}
```

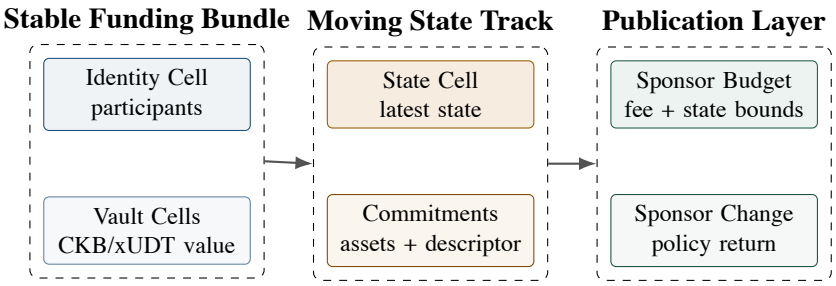
The signed message is

$$m_n = \mathsf{H}(\texttt{"CKB\_MORPH\_CHANNEL\_STATE\_V1"} \parallel \mathrm{canonical}(\mathsf{Header}_n)).$$

The version fields, funding epoch, funding anchor, and vault-set commitment are part of the signed domain. They prevent replay across channels and prevent an old signature from being reinterpreted by a later descriptor, funding anchor, vault set, signature scheme, or state-layout parser.

5.4 Sponsor Partition

Sponsor funds are not part of the channel-owned value partition. They are standard wallet Cells, watchtower-owned Cells, or pre-authorised fee budgets. The script-enforced budget boundary should include channel identity, state-number range, maximum fee, expected state type, and uncontaminated change destination. Expiry windows, allowed sponsor sources, publishing cadence, and webhook policy may also be enforced by an operator or watchtower layer. If a deployment lacks script-verifiable evidence for such a field, the field must not be silently treated as a script-enforced safety condition.

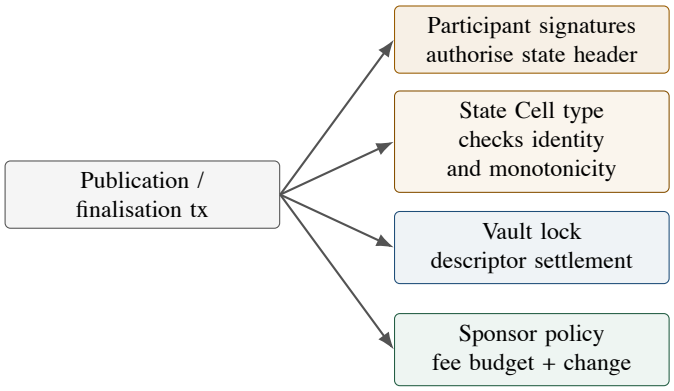


The state track binds the funding identity; the publication layer meets carrier-transaction costs. Vault value is not used for fees.

Figure 2: Protocol object map. The funding bundle holds channel identity and value; the state track carries authority; the publication layer meets the transaction fee.

6 Script-Level Validation Semantics

The following pseudo-code states the script checks required for an implementable construction. It is not byte-level CKB-VM code.



State evidence does not by itself spend value; sponsor authorisation does not by itself authorise channel value.

Figure 3: Authorisation boundaries. State signatures, vault spends, and sponsor fee authorisation are separate checks.

6.1 State Cell Type Semantics

The State Cell type script owns state progression. It does not spend channel value; it verifies the identity and monotonicity of state evidence. Ordinary publication and supersession preserve the current funding epoch, funding anchor, and vault-set commitment. A splice is a separate re-anchoring transition: it must prove both the old and new funding context, rather than silently changing these fields inside an ordinary state update.

```
verify_state_transition(tx):
  old = exactly_one_input_state_cell(tx, channel_id)
  new = exactly_one_output_state_cell(tx, channel_id)
  old_header = parse_state_header(old.data)
  new_header = parse_state_header(new.data or witness)

  require new_header.protocol_version == old_header.protocol_version
  require new_header.chain_id == old_header.chain_id
  require new_header.signature_scheme_id
```

```
      == old_header.signature_scheme_id
  require new_header.channel_id == old_header.channel_id
  require new_header.funding_epoch == old_header.funding_epoch
  require new_header.funding_anchor_identity
      == old_header.funding_anchor_identity
  require new_header.vault_set_commitment
      == old_header.vault_set_commitment
  require new_header.asset_registry_commitment
      == old_header.asset_registry_commitment
  require descriptor_transition_admissible(
      old_header.settlement_descriptor_commitment,
      new_header.settlement_descriptor_commitment,
      old_header.descriptor_version,
      new_header.descriptor_version,
      witness)
  require new_header.challenge_policy_commitment
      == old_header.challenge_policy_commitment
  require new_header.state_number > old_header.state_number
  require new_header.phase == settling

  require referenced_funding_anchor(tx.cell_deps)
      matches new_header.funding_anchor_identity
  require signatures_valid(new_header, witness.signatures)
  require payload_or_proof_matches_commitment(new_header, witness)
  require partition_conservation(tx, new_header.asset_registry_commitment)
  require output_state_capacity_sufficient(new)
```

The descriptor transition predicate is narrow. It permits a new descriptor instance only when the new State Header is signed under the normal state-signing domain and the descriptor version and grammar remain within the protocol's accepted boundary. The vault later settles only against the descriptor commitment in the current authentic State Cell.

Splice validation is separate from ordinary monotonic publication. It must bind the old funding anchor, the new funding anchor, the old vault-set commitment, the new vault-set commitment, the asset deltas, and the participant signatures in one signed re-anchoring event. This prevents a watchtower package or settlement descriptor from an earlier funding epoch from being replayed after the channel has been resized.

The "exactly one input, exactly one output" rule is the script-level expression of State Cell uniqueness. Mempool races are resolved by the ordinary CKB single-spend rule: competing transactions may exist, but at most one can consume the current State Cell on-chain.

6.2 Vault Lock Semantics

Vault locks own channel value. They must not accept a transaction merely because the transaction references the correct channel. They accept only settlement or materialisation authorised by the current state evidence.

```
verify_vault_spend(tx):
  op = classify_channel_operation(tx)
  require op in {FINALIZE, COOPERATIVE_CLOSE, SPLICE, MATERIALIZE}

  state = exactly_one_authentic_input_state_cell(
         tx, channel_id, expected_state_type, expected_state_lock)
  header = parse_state_header(state.data)
  require header.funding_anchor_identity matches this_vault.channel
  require signatures_or_phase_authorise(op, header, tx.witness)

  if op == FINALIZE:
```

```
  require header.phase == settling
  require state_input_relative_block_since_satisfies(header.challenge_policy)
  require this_vault matches header.payload_or_vault_commitment

 expected_outputs =
  derive_outputs(header.settlement_descriptor_commitment,
              header.payload_commitment,
              tx.witness)
 require canonical_outputs(tx.channel_outputs) == expected_outputs
 require partition_conservation(tx, header.asset_registry_commitment)
```

This lock is the point at which "current valid state" becomes economically meaningful. A State Cell without vault enforcement is only evidence; it does not protect value.

6.3 Bounded Sponsor Budget Semantics

A sponsor budget may be implemented as a standard wallet signature, a watchtower-owned Cell, or a bounded policy Cell. The bounded policy form is the most delicate because it authorises another actor to spend capacity for publication.

```
verify_sponsor_budget_spend(tx):
 policy = parse_policy(this_budget_cell.data)
 header = parse_published_state_header(tx.witness)

 require header.channel_id == policy.channel_id
 require policy.min_state_number <= header.state_number
 require header.state_number <= policy.max_state_number
 require tx_fee(tx) <= policy.max_fee_per_tx
 require cumulative_fee_after(tx) <= policy.max_total_fee
 require sponsor_change_lock(tx) == policy.change_lock
 require no_channel_assets_in_sponsor_change(tx)
 require operation_is_publication_or_challenge(tx)
```

Expiry windows, allowed sponsor sources, scan cadence, and webhook policy remain operator or watchtower policy unless the deployment provides the script with verifiable evidence for those fields. A bounded sponsor script rejects, or otherwise makes inadmissible, a finite script-level deadline for which it cannot verify the relevant chain evidence.

The policy is not an open wallet delegation. It is a bounded publication right scoped to one channel and one state interval.

7 Protocol State Machine

Let the channel phase be one of

$$\mathsf{Phase} = \{\mathsf{funding}, \mathsf{active}, \mathsf{settling}, \mathsf{closed}\}.$$

The lifecycle contains six operations:

$$\mathsf{FUND},\quad \mathsf{OFFCHAIN\_UPDATE},\quad \mathsf{PUBLISH},\quad \mathsf{SUPERSEDE},\quad \mathsf{SPLICE},\quad \mathsf{FINALIZE}.$$

8 Transaction Semantics

8.1 FUND

FUND creates the stable funding bundle and the initial State Cell. The funding transaction may be funded by standard wallet inputs. After funding, channel-owned value enters the channel partition and should not be used for publication fees.

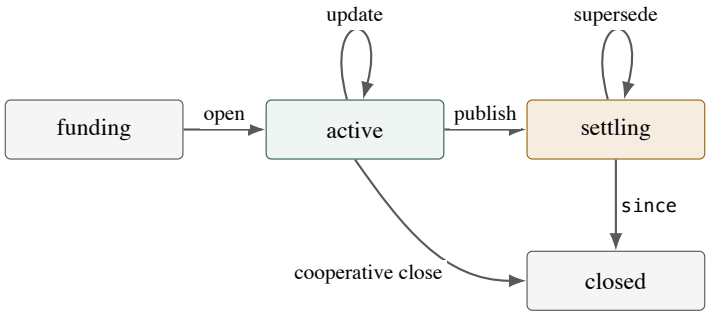


Figure 4: Morph Channel state machine. Publishing moves an active channel into the settling phase; newer signed evidence supersedes older settling evidence before finalisation.

The initial State Cell is the only on-chain active pointer created by the channel. Its type arguments or equivalent identity mechanism must bind the same funding-anchor identity used by the Fund Cell and vault locks. A transaction that attempts to create two initial State Cells for the same channel genesis identity is invalid under the funding script profile.

8.2 OFFCHAIN_UPDATE

An off-chain update produces a new state evidence object with a strictly larger state number. It does not consume CKB Cells. Participants exchange signatures over the canonical state header. State numbers are strictly monotonic, but not necessarily consecutive: a valid supersession may move from n to $n + k$ if the newer state is properly authorised and the sponsor policy admits that state interval.

Off-chain active updates do not create on-chain active State Cells. Once a unilateral publication reaches the chain, the channel is in the settling phase. Further on-chain progress is either supersession by a newer settling State Cell or finalisation after the relative challenge delay.

8.3 PUBLISH and SUPERSEDE

The same transaction pattern handles initial unilateral publication and challenge:

```
PUBLISH_STATE:
  inputs:
    StateCell(n, active_or_settling)
    SponsorCell*
  cell_deps:
    FundingAnchor
    VaultCell*
  outputs:
    StateCell(n', settling)
    SponsorChange*
  validity:
    n' > n
    signatures over canonical Header(n') are valid
    funding identity matches referenced anchor
    channel partition is conserved
```

Participant signatures authorise the state header. The transaction body selects live inputs, sponsor inputs, and fee rate, so a challenger may rebuild it against the currently live State Cell under competing tx-pool spends.

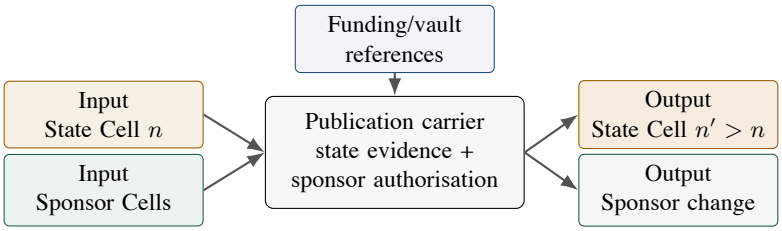


Figure 5: Publication and supersession transaction shape. The channel evidence can be reused while the sponsor inputs and fee choice are rebuilt.

8.4 SPLICE

SPLICE resizes a channel without closing its logical relationship. It is neither an ordinary off-chain update nor a final settlement. It is a signed re-anchoring transition that consumes the current active state and old vault authority, advances the funding epoch, and creates a new state and vault set for the same logical channel.

A splice-in increases the channel-controlled asset set by admitting external value. A splice-out releases value while preserving enough post-splice vault value to satisfy the latest signed settlement descriptor. In both cases, the signed transition must bind the old funding anchor, the new funding anchor, the old vault-set commitment, the new vault-set commitment, and the per-asset deltas. CKB and xUDT lanes are checked by asset identity and amount, not by informal wallet labels.

The freshness consequence is material for recovery. A watchtower package saved before a splice is not automatically valid after the splice, even if it has a higher state number than some local record, because its signed funding epoch and vault-set commitment refer to a different enforceable value object.

8.5 FINALIZE

FINALIZE consumes the current settling State Cell and the relevant vault Cells after the relative `since` delay has matured. It creates the settlement outputs authorised by the committed settlement descriptor. Retiring the state track without finalising or otherwise leaving verifiable finalisation evidence is invalid, because it would orphan channel value. Cooperative close is a special path that carries explicit authorisation from the required participants and does not require a challenge delay.

9 Partition Conservation

For every post-funding transaction, partition the transaction's values into channel-owned value C and sponsor-owned value S. Read-only `cell_deps` do not participate in value accounting. The high-level rule is:

$$C_{\text{in}} = C_{\text{out}}, \qquad S_{\text{in}} - S_{\text{out}} = \text{fee}, \qquad C \cap S = \emptyset.$$

Invariant 1 (Partition Conservation)**.** *Channel-owned value is exactly conserved across post-funding publication and challenge transactions. Any fee deficit is borne exclusively by the sponsor partition.*

9.1 Capacity-Aware Conservation Algorithm

CKB capacity requires a finer rule than a single scalar balance. A channel-owned CKB Cell has two logically distinct components:

$$\mathsf{capacity}(c) = \mathsf{reserve}(c) + \mathsf{business\_ckb}(c), \qquad \mathsf{reserve}(c) \geq \mathsf{occupied}(c).$$

The reserve component exists so the Cell can exist. The business component is user-visible CKB value. A valid transition must not silently convert reserve into fee, sponsor change, or business balance unless the descriptor explicitly authorises a reserve refund.

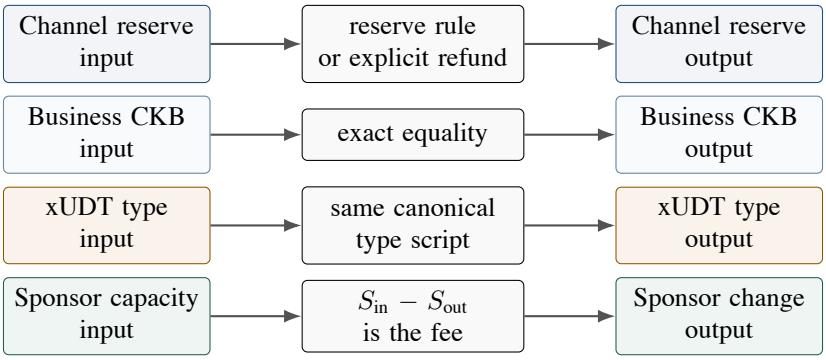


Figure 6: Partition conservation. Capacity needed for storage, user-visible CKB, each xUDT type, and sponsor fees are checked as separate lanes.

For xUDT accounting, an asset type is the canonical identity of the full type script, for example the hash of its canonical encoding. It is not a symbol, name, display metadata, or wallet-level asset label.

```
partition_conservation(tx, asset_registry):
  classify every input/output cell as:
    SPONSOR
    CHANNEL_RESERVE
    BUSINESS_CKB
    BUSINESS_XUDT(asset_type)
    STATE_CARRIER
    UNRELATED

  require UNRELATED cells are not read by channel scripts
        and do not contribute to channel conservation
  require all channel output cells satisfy
        capacity(output) >= occupied_capacity(output)

  reserve_in = sum(channel_reserve_amount(input))
  reserve_out = sum(channel_reserve_amount(output))
  require reserve_out + authorised_reserve_refund == reserve_in

  business_ckb_in = sum(business_ckb_amount(input))
  business_ckb_out = sum(business_ckb_amount(output))
  require business_ckb_out == business_ckb_in

  for each asset_type in asset_registry.xudt_types:
    type_script = asset_registry.type_script(asset_type)
    require asset_type == hash(canonical(type_script))
    require sum_xudt(input, type_script)
          == sum_xudt(output, type_script)
    require every xUDT output for asset_type
          carries type_script

  require sponsor inputs are not classified as channel reserve
  sponsor_in = sum_capacity(sponsor_inputs)
  sponsor_out = sum_capacity(sponsor_outputs)
  require sponsor_in - sponsor_out == tx_fee(tx)
  require sponsor_outputs carry no channel business assets
  require sponsor_outputs carry no registered xUDT type
  require sponsor_outputs do not use channel vault locks
        unless explicitly classified by the descriptor
```

The algorithm separates four questions that are otherwise readily conflated: whether a Cell can ex-

Partition	Source	Role	Fee payer
Sponsor capacity	standard wallet or budget Cells	transaction fees and change	yes
Channel reserve	channel-owned CKB capacity	Cell occupancy and reserve refund	no
Business assets	CKB or xUDT balances	participant settlement value	no

Table 1: Value partitions in Morph Channel.

ist, whether channel reserve is conserved or refunded, whether CKB business value is conserved, and whether each xUDT type is conserved under its own type script. Unrelated Cells may appear only when they are irrelevant to channel script reads and irrelevant to conservation. They must not be used as hidden witnesses for channel authority.

Proposition 1 (Zero Channel-Paid Publication Fees)**.** *If partition conservation holds for a post-funding transaction, then publication fees cannot reduce channel-owned value.*

Proof. CKB transaction fees are represented by the difference between input capacity and output capacity. Since channel reserve, business CKB, and each registered xUDT type are conserved within the channel partition, no fee deficit can be charged to channel-owned value. The only permitted capacity deficit is $S_{\text{in}} - S_{\text{out}}$, the sponsor partition. □

10 Settlement Descriptor

The settlement descriptor is a protocol object, not a general-purpose application language. It commits to the admissible output derivation rule for settlement and local materialisation. A minimal descriptor specifies:

(1) participant destination policies;

(2) per-asset allocation rules;

(3) occupied-capacity requirements for outputs;

(4) reserve refund rules;

(5) required xUDT type scripts;

(6) sponsor change classification;

(7) descriptor version and upgrade boundary.

Invariant 2 (Descriptor Boundedness)**.** *The descriptor must authorise settlement, splice, factory materialisation, and reserve refund. It must not introduce unbounded application execution into channel witnesses.*

Invariant 3 (Exact Materialisation)**.** *A signed descriptor or vault delta is not only an accounting statement. For every value-bearing Cell it authorises, the realised Cell must match the committed lock identity, capacity, type-script identity, and data-encoded amount. A proof that only one asset lane changed is insufficient if the resulting Cell shape can drift in an untouching lane.*

11 Bilateral Witness Mode

For a two-party channel, the first witness format should remain plaintext. Bilateral state is typically small: balances, conditional payments, timeout information, and shutdown policies. Plaintext witnesses keep force-close transactions auditable and make early prototypes simpler to test. Privacy may later

be improved by replacing the payload with committed state material once size and CKB-VM costs are measured.

Plaintext is not authority by itself. The authoritative object remains the signed state header and its payload commitment. A validator checks that the plaintext payload deterministically induces the committed payload hash and satisfies the descriptor and conservation rules.

12 Factory Proof Mode

Factories require a different authority model. A factory may contain many participants, balances, subchannels, reserves, and local exit paths. Full plaintext disclosure on every local exit exposes unrelated participant data and may cause witness size and verification cost to grow with factory size. Factory authority should therefore be commitment-first.

Definition 7 (Factory State)**.** *A factory state is a state header plus a set of roots:*

```
FactoryState {
  state_header
  balances_root
  subchannels_root
  membership_root
  reserve_root
}
```

Definition 8 (Factory Transition Envelope)**.** *A factory transition is admissible only after a bounded outer claim identifies its transition family and commits to exactly one evidence body:*

```
FactoryEnvelope {
  transition_family
  body_bound
  body_commitment
}
```

The envelope is not the local proof. It is the admission-control object that specifies which proof grammar is being invoked and which evidence body is committed. A body that merely parses as some factory payload is not authority unless the outer claim admits that transition family and binds that body.

Definition 9 (Factory Witness)**.** *A factory witness contains an outer transition envelope, an access manifest, a proof bundle, touched leaf payloads, and the required signatures:*

```
FactoryWitness {
  envelope
  state_header
  access_manifest
  proof_bundle
  touched_leaf_payloads
  signatures
}
```

Definition 10 (Transition Footprint)**.** *The transition footprint of a factory action is the set of leaves read, written, inserted, removed, or proven absent by that action.*

Definition 11 (Factory Right)**.** *A factory right is any participant-relevant claim that may be changed by a local factory action:*

$$\mathsf{FactoryRight} = \{\mathsf{balance}, \mathsf{reserve\_claim}, \mathsf{membership}, \mathsf{exit\_path}, \mathsf{sponsor\_budget\_claim}\}.$$

Definition 12 (Non-Interference)**.** *A factory action is non-interfering with respect to untouched participants if the proof bundle establishes that all their factory rights are unchanged, including rights that depend on shared reserve, membership, exit path, or public sponsor-budget state.*

A Merkle proof is insufficient unless the transition family also accounts for rights depending on reserve, membership, exit path, and sponsor-budget state. A touched balance leaf may also affect reserve liquidity, local exit eligibility, or sponsor-budget allocation. A proof of non-interference must either cover those dependencies or the action must fall back to full-participant authorisation. An envelope-first boundary prevents a reduced proof from being checked under a weaker authority rule.

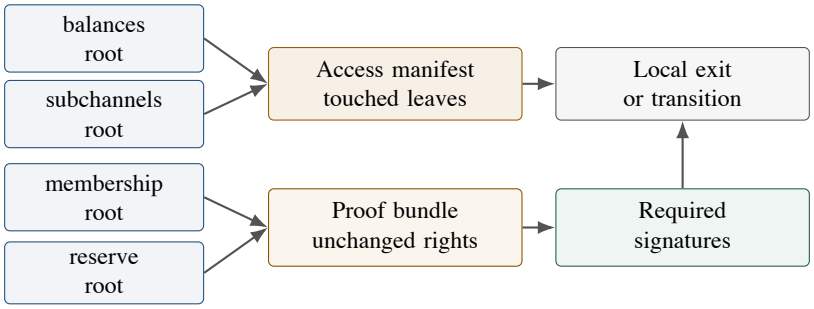


Figure 7: Factory proof mode. A local transition must expose the leaves it touches and prove that the relevant untouched rights are unchanged.

Proposition 2 (Conservative Factory Authorisation)**.** *If a factory action cannot prove non-interference, then a conservative protocol profile should require signatures from all factory participants. If non-interference is proven, the signing set may be reduced to the affected participants only when the touched rights also satisfy participant authorisation, local validity, and value conservation.*

Proof. Without non-interference, untouched leaves may still depend on shared membership, reserve, or exit-right state. A local signature set would not authorise those possible effects. Full-participant signatures authorise global effects. A non-interference proof removes only the effects on untouched rights; it does not by itself authorise minting a balance, increasing a reserve claim, deleting membership, or releasing value from a vault. □

Proposition 3 (Envelope-First Factory Admissibility)**.** *If the factory transition family is not fixed and committed before the local proof body is interpreted, then reduced authorisation is unsafe.*

Proof. Different factory paths authorise different effects: a conservative update, a local exit, a single-right reduction, a sparse update, or a splice all have different admissible touched rights and value consequences. If the proof body determines its own authority class, an adversary may seek an alternative interpretation with weaker checks. A bounded outer claim fixes the transition family and commits to the body before local validity is evaluated, so the proof is checked under the intended authority rule only. □

13 Factory Acceptance Requirements

A deployment is not accepted merely because one representative factory path succeeds. A candidate deployment must pass an end-to-end devnet acceptance matrix in which every value-bearing factory flow is exercised as a committed transition, and every attack-shaped factory rejection is checked against its intended error class. The matrix is stated at protocol level without prescribing a transaction builder, storage format, or proof encoding.

Factory flow family	Required positive acceptance evidence	Required rejection evidence
Factory lifecycle	factory creation; all-participant state advance; local child-channel materialisation; child publication; child finalisation	stale, duplicate, or unauthorised state authority must not become value authority
Reduced rights	bounded right decrease; tight near-zero decrease; sparse single-right update	a locality proof must not authorise value creation or unrelated-right mutation
CKB reduced exit	reserve-claim release into a local child channel; asymmetric participant capacity distribution	released CKB and remaining reserve must match the authorised descriptor exactly
Typed reduced exit	partial xUDT release with factory-vault change; full xUDT release; one-sided xUDT settlement	mismatched typed settlement output or factory-vault typed change must be rejected
Conservative factory splice	CKB splice-in and splice-out; asymmetric CKB capacity variants; xUDT splice-in and splice-out; one-sided xUDT variants	vault descriptors and per-asset deltas must bind the realised Cell shape, not merely the aggregate amount
Reduced factory splice	CKB reduced splice-in and splice-out; asymmetric CKB variants; xUDT reduced splice-in and splice-out; one-sided xUDT variants	sparse locality evidence must be checked under the admitted transition family only
Negative recovery continuity	a valid later factory or child-channel transition remains publishable after an expected rejection	attack-shaped typed materialisation failures must reject without poisoning later progress

Table 2: Factory acceptance matrix. Every row is a required devnet acceptance family for a candidate deployment of the factory protocol.

The acceptance rule is strict: every positive row in Table 2 must be represented by committed on-chain evidence, and every negative row must be represented by an exact rejection class. The non-interference proof must be accompanied by acceptance evidence showing that conservative paths, reduced paths, CKB paths, typed-asset paths, asymmetric value distributions, and one-sided typed distributions all preserve the same authority and conservation boundaries.

14 Challenge-Window Semantics

The challenge window is a deployment parameter derived from CKB confirmation behaviour and watchtower liveness.

Let Δ denote the finalisation delay encoded by relative `since`. A conservative policy should satisfy:

$$\Delta \geq \Delta_{\text{detect}} + \Delta_{\text{poll}} + \Delta_{\text{build}} + \Delta_{\text{confirm}} + \Delta_{\text{margin}}.$$

The terms respectively represent detection confirmations, watchtower polling interval, transaction construction and propagation, confirmation of the newer-state transaction, and a margin for fee bumping and reorganisation risk. The current conservative profile uses a canonical relative block-number `since`. A future deployment may choose epoch- or timestamp-based maturity only if the mode, metric, reserved bits, and value are canonicalised and tested against current CKB hardfork semantics [8, 9, 10]; raw integer comparison is not a valid challenge-window check.

Parameter	Deployment evidence	Audit question
Detection depth	chosen confirmation depth, for example 1, 3, or 6 confirmations	when does a watchtower treat stale evidence as actionable?
Polling interval	watchtower service objective	is there enough time between observation and finalisation?
Build and broadcast time	wallet, RPC, and network measurements	can a replacement carrier be assembled against the live State Cell?
Confirmation target	measured CKB confirmation behaviour	can the newer state confirm before the deadline?
Fee-bump margin	sponsor budget multiple	is there budget for conservative publication plus emergency rebuild?
Reorganisation margin	conservative deployment choice	does the delay tolerate expected reorg risk?

Table 3: Deployment profile required for selecting a challenge window.

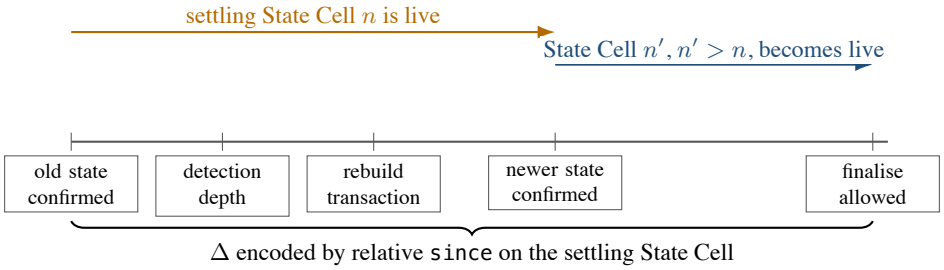


Figure 8: Challenge-window timeline. Safety depends on confirmed supersession before the relative finalisation delay matures, not on tx-pool visibility alone.

14.1 CKB `since` and Mempool Reality

In CKB, relative `since` is checked on transaction inputs. For Morph finalisation, the relevant input is the currently live settling State Cell. Therefore a finalisation transaction should set `since` on the State Cell

input and the State type or vault lock should check that the value is an encoded relative-block condition matching the committed challenge policy. A naked numeric value is not equivalent to a relative delay.

The safety condition is confirmation before the deadline, not mempool presence. A challenger may observe an old state in the tx-pool, but the protocol need not respond until an old settling State Cell is actually confirmed or reaches the configured detection depth. If the old state confirms, the challenger rebuilds a newer-state transaction against the now-live settling State Cell. If a newer-state transaction remains unconfirmed, the watchtower must be able to rebuild it with different sponsor inputs or a higher fee, because the channel signature does not commit to sponsor coin selection.

Fee bumping does not rely on Bitcoin-style replace-by-fee semantics. It means reconstructing a fresh transaction body against the current live State Cell and current sponsor funds. Once a competing transaction consumes that State Cell, all stale transaction bodies become invalid and must be rebuilt.

```
watchtower_response_loop:
  observe chain tip and tx-pool
  if confirmed settling StateCell(n) reaches detection_depth:
    latest = local_state_package(channel_id)
    if latest.state_number > n:
      tx = build_publish_state_tx(
           input_state = live StateCell(n),
           state_package = latest,
           sponsor_policy = available_budget(),
           fee_rate = conservative_fee_rate())
      broadcast(tx)
      while tx not confirmed and finalisation_deadline not near:
        if fee market or sponsor set changes:
          tx = rebuild_against_live_state_cell()
          broadcast(tx)
```

The watchtower policy must reserve enough fee budget for at least one conservative publication and one emergency rebuild. If the deployment expects volatile fees, the budget should support additional rebuild attempts. This is an operational requirement.

Proposition 4 (Newer-State Supersession)**.** *Suppose a settling State Cell at number n is live. If an honest party publishes and confirms valid evidence $E_{n'}$ with $n' > n$ before finalisation of E_n, then E_n cannot be the final settlement authority.*

Proof. The supersession transaction consumes the live settling State Cell at n and creates a new settling State Cell at n'. Once consumed, the n State Cell cannot be used as an input to finalisation. The finalisation rule applies only to the currently live settling State Cell after its own relative `since` delay matures. □

15 Signature Domains and Authorisation Boundaries

Morph separates three authorisation domains:

(1) state authorisation: participant signatures over the canonical State Header;

(2) value authorisation: vault locks accepting only descriptor-authorised settlement or materialisation;

(3) fee authorisation: sponsor signatures or bounded sponsor policies.

Lemma 1 (Domain-Separated Anti-Replay)**.** *If protocol version, signature-scheme identity, chain identity, channel identity, funding epoch, funding anchor identity, vault-set commitment, asset registry, descriptor commitment, descriptor version, challenge policy, and state number are included in the signed State Header, then a valid state signature cannot be replayed as a valid state for another channel context, funding context, or later protocol interpretation.*

Proof. Any change to the protocol version, signature scheme, chain, channel, funding epoch, funding anchor, vault set, asset registry, descriptor, descriptor version, challenge policy, or state number changes the canonical State Header and hence the signed message. Under signature unforgeability, the adversary cannot produce a valid participant signature for the modified context. □

16 Watchtower and Recovery Objects

A watchtower stores the latest state package and publishes newer evidence when stale evidence appears on-chain; it never receives authority over channel-owned value. A state package contains more operational material than a bare root, but less disclosure than a permanent plaintext dump:

$$\mathsf{StatePackage} = (\mathsf{Header}, \mathsf{Payload}\text{ or roots}, \Pi, \Sigma, \mathsf{sponsor\_policy}).$$

The script-enforced sponsor boundary must be bounded by channel identity, state-number range, maximum fee, expected State Type, and change destination. Expiry windows, allowed sponsor sources, and publishing cadence are operator or watchtower policy in the conservative profile unless a deployment provides script-verifiable evidence for those fields; finite script-level deadlines without such evidence should be rejected rather than treated as consensus-enforced policy. The watchtower must not be able to redirect channel-owned value or drain arbitrary wallet funds.

After a splice, watchtower recovery must be funding-context aware. A stored package is publishable only if its signed funding epoch, funding anchor, and vault-set commitment match the currently live state track. Otherwise it is merely historical evidence for an older funding context.

17 Safety Properties

Proposition 5 (Funding Context Uniqueness)**.** *If the State Header binds the funding epoch, funding anchor identity, and vault-set commitment, and scripts verify that referenced* `cell_deps` *match that context, then a descriptor-shaped but non-authoritative Cell cannot serve as the channel's funding anchor or vault set.*

Proof. The script compares the referenced Cell identity and vault-set commitment with the context committed in the State Header. A different Cell, even with similar data, has a different outpoint, genesis identity, or vault-set commitment and is rejected. □

Proposition 6 (Post-Splice Freshness)**.** *If a splice advances the funding epoch and commits to a new vault set, then a state package from an earlier funding epoch cannot authorise settlement of the post-splice vaults.*

Proof. The earlier package signs a different funding epoch, funding anchor, or vault-set commitment. Since vault settlement requires the authentic current State Cell and the descriptor bound to that funding context, the older package does not match the post-splice value object. □

Proposition 7 (Vault Spend Safety)**.** *If each vault lock accepts only settlement or materialisation transactions authorised by an authentic current State Cell and descriptor-admissible outputs, then channel-owned value cannot be spent through standard wallet paths or fake state-header bytes.*

Proof. Every spend of channel-owned value must satisfy the vault lock. By assumption, the lock rejects transactions lacking authentic current state evidence and descriptor-admissible outputs. Ordinary wallet signatures alone are insufficient, and a non-authoritative Cell that only imitates the State Header layout does not satisfy the state-evidence predicate. □

Proposition 8 (Recoverable Unilateral Settlement)**.** *If a participant or watchtower stores a valid State Package, has access to sponsor funds within policy, and the challenge delay satisfies the response-budget inequality, then it can reconstruct a publication transaction without fresh counterparty cooperation.*

Proof. The channel signature excludes sponsor inputs and fee rate, so the publication transaction may be rebuilt using available sponsor funds. The State Package supplies the signed state evidence and required witness or proof. The response-budget inequality provides time for detection, construction, propagation, and confirmation. □

18 Evaluation Agenda

Empirical closure requires the following benchmark evidence:

E1. bilateral plaintext witness size under realistic payment states;

E2. factory proof size under realistic touched-footprint distributions;

E3. CKB-VM cycles for signature verification, hash verification, proof verification, and xUDT type-script composition;

E4. challenge reliability under confirmation-depth, fee-volatility, and watchtower-polling parameters;

E5. sponsor-policy failure cases, including insufficient budget, inadmissible finite script-level expiry, operator-level expiry, and wrong change destination;

E6. State Cell uniqueness tests under concurrent publication and supersession attempts;

E7. splice freshness tests showing that packages from earlier funding epochs cannot govern later vault sets;

E8. vault-lock rejection tests for spends lacking current state evidence;

E9. reserve refund, occupied-capacity, business-CKB, and per-xUDT conservation edge cases;

E10. factory envelope-admission tests for transition-family confusion, body-bound violations, and digest mismatch;

E11. factory acceptance evidence for every lifecycle, reduced-rights, reduced-exit, conservative-splice, reduced-splice, asymmetric, and one-sided typed path in Table 2;

E12. executable negative tests corresponding to the audit matrix in Appendix A;

E13. comparison between full plaintext factory disclosure and local proof verification.

19 Related Work and Positioning

Lightning [1] represents the penalty-based payment-channel family. Eltoo [2] provides the update-by-replacement intuition, but depends on Bitcoin sighash changes. Channel factories [3] motivate shared funding and local exit questions. State-channel frameworks such as Perun [4] show the broader adjudication pattern in which off-chain evidence is made enforceable by an on-chain dispute procedure. CKB community work on generic payment channels [5] is a nearer point of comparison, because it studies channel composability directly on the Cell model.

CKB's Cell model [6] and transaction structure [7] provide explicit state objects and read-only dependency references. CKB's `since` field [8, 9, 10] provides relative lock semantics needed for challenge windows. SUDT [11] and xUDT [12] motivate per-asset conservation rather than a single undifferentiated balance.

Morph occupies a narrower point in this design space: CKB can host a channel object whose funding identity, state evidence, fee publication, and factory-local proof objects are separated at the script level without changing base-layer consensus.

Work	Funding and state shape	Dispute rule	Fee model	Morph distinction
Lightning	funding output plus commitment transactions	stale states are penalised	fees are carried by commitment or closing transactions	separates State Cell evidence from vault value
eltoo	replacement-style update state over a funding output	newer state replaces older state	depends on transaction form and proposed sighash support	implements newer-state supersession with current CKB scripts
Channel factories	shared funding for many off-chain relationships	local exits depend on factory validity rules	not the central object	adds envelope-first local proof admission and non-interference tests
Perun	adjudicated state-channel framework	on-chain dispute procedure enforces off-chain evidence	framework-dependent	specialises the adjudication object to CKB Cells and vault locks
CKB generic channels	Cell-based channel composability	CKB script enforcement	CKB transaction fees	adds sponsor partitioning, splice freshness, and per-asset conservation

Table 4: Positioning against related channel constructions.

20 Limitations and Open Questions

The construction remains incomplete in several respects:

O1. Factory coordination remains an off-chain protocol problem.

O2. Proof mode may not be cheaper than plaintext until measured.

O3. Mainnet challenge delays require empirical confirmation and fee data.

O4. Descriptor versioning must balance upgrade flexibility against recoverability.

O5. Different xUDT variants may impose different constraints.

O6. The paper now specifies protocol-level validation semantics, but not every deployment-level encoding or operational procedure.

O7. Factory non-interference still requires a formal rights-dependency schema before reduced signing sets are safe.

O8. Factory envelope families and proof-size bounds require continued measurement under realistic transition mixes.

O9. Typed reduced exits and typed factory vault changes are admissible only when asset type, amount, capacity, child or factory vault Cell shape, settlement descriptor, and factory-vault change are completely bound. Broader typed-asset compatibility remains future work.

21 Conclusion

Morph Channel sets out how to separate funding identity, state evidence, sponsor fee responsibility, and factory-local proof admission using current CKB script objects. The construction keeps channel value in

vault Cells, moves dispute authority through a State Cell, checks channel and sponsor value in separate partitions, and requires factory-local proofs to be admitted by a bounded transition envelope.

The remaining burden is empirical and operational. A deployable protocol requires benchmark evidence for witness size, CKB-VM cycles, challenge-window parameters, fee-budget policy, splice recovery, and factory proof costs. Until those measurements exist, Morph remains a script-level construction for CKB channels rather than a production channel network.

A Audit Matrix

This appendix states the construction audit target in testable form. Each row should become at least one positive test and one negative test in any candidate deployment.

References

[1] Joseph Poon and Thaddeus Dryja. The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments. https://lightning.network/lightning-network-paper.pdf.

[2] Christian Decker, Rusty Russell, and Olaoluwa Osuntokun. eltoo: A Simple Layer2 Protocol for Bitcoin. https://blockstream.com/eltoo.pdf.

[3] Conrad Burchert, Christian Decker, and Roger Wattenhofer. Scalable Funding of Bitcoin Micropayment Channel Networks. https://tik-old.ee.ethz.ch/file/
/49218d6b9325ddb4f18aef8830ec567b/1/scalable_funding.pdf.

[4] Stefan Dziembowski, Lisa Eckey, Sebastian Faust, and Daniel Malinowski. Perun: Virtual Payment Hubs over Cryptocurrencies. Cryptology ePrint Archive, Report 2017/635. https://eprint.iacr.org/2017/635.

[5] Nervos Talk. A Generic Payment Channel Construction and Its Composability. https://talk.nervos.org/t/
a-generic-payment-channel-construction-and-its-composability/4697.

[6] Nervos RFC 0002. CKB Cell Model. https://nervosnetwork.github.io/rfcs/rfcs/0002-ckb/0002-ckb.html.

[7] Nervos RFC 0022. CKB Transaction Structure. https://nervosnetwork.github.io/rfcs/
rfcs/0022-transaction-structure/0022-transaction-structure.html.

[8] Nervos RFC 0017. Transaction valid since. https://nervosnetwork.github.io/rfcs/
rfcs/0017-tx-valid-since/0017-tx-valid-since.html.

[9] Nervos RFC 0028. Change since relative timestamp. https://github.com/nervosnetwork/
rfcs/blob/master/rfcs/0028-change-since-relative-timestamp/
0028-change-since-relative-timestamp.md.

[10] Nervos RFC 0030. Ensure index is less than length in since. https://github.com/nervosnetwork/rfcs/blob/master/rfcs/
0030-ensure-index-less-than-length-in-since/
0030-ensure-index-less-than-length-in-since.md.

[11] Nervos RFC 0025. Simple UDT. https:
//nervosnetwork.github.io/rfcs/rfcs/0025-simple-udt/0025-simple-udt.html.

[12] Nervos Talk. RFC: Extensible UDT. https://talk.nervos.org/t/rfc-extensible-udt/5337.

ID	Invariant	Enforced by	Positive case	Negative case
S1	One live State Cell controls the channel pointer	State Cell type plus CKB single-spend	supersede $n \to n+1$ by consuming current State Cell	transaction creates two successor State Cells
S2	Funding anchor identity is canonical	State Header plus `cell_deps` check	referenced anchor matches committed genesis identity	descriptor-shaped fake anchor is referenced
S3	State numbers are strictly monotonic	State Cell type script	supersede $n \to n+k$ with valid signatures	publish lower or equal state number
S4	State retirement does not orphan value	State and vault authority coupling	retirement finalises value or leaves verifiable finalisation evidence	State Cell is consumed while matching vault value remains live without evidence
S5	Splice advances funding context without changing channel identity	signed funding epoch and vault-set commitment	post-splice state binds new anchor and vault set	pre-splice package controls post-splice vaults
V1	Vault value follows current state evidence	Vault lock and settlement descriptor	finalise after valid `since` delay	wallet-only spend of vault value
V2	Vault authority requires an authentic State Cell	State/Vault script identity checks	State Cell under expected scripts controls finalise	ordinary Cell carries decodable State Header bytes
V3	Descriptor evolution is signed state semantics	State signature and descriptor commitment	newer signed state commits to an admissible descriptor instance	finalise supplies an unsigned replacement descriptor
P1	Channel-owned capacity never pays publication fees	partition conservation	sponsor capacity pays fee and receives change	reserve reduced by transaction fee
P2	Reserve and business CKB are not confused	reserve and business-CKB ledgers	authorised reserve refund is explicit	reserve becomes hidden business balance or fee
X1	xUDT value is conserved by canonical type script	asset registry and xUDT type check	same canonical type script and amount	same symbol or metadata with different type script
G1	Sponsor change is uncontaminated	sponsor policy and partition classifier	sponsor change contains only sponsor capacity	sponsor change carries registered xUDT or vault lock
G2	Unverifiable sponsor deadlines are not script policy	sponsor policy boundary	unbounded script policy or verifiable deadline evidence	finite deadline is accepted without chain-verifiable evidence
U1	Unrelated Cells cannot influence channel validity	script read set and conservation classifier	unrelated wallet input pays no channel role	helper Cell is read by channel scripts without classification
C1	Challenge safety is confirmation-based	deployment profile and `since` policy	newer state confirms before deadline	stale state finalises while newer tx is only in mempool
C2	Challenge maturity is canonical relative block `since`	State/Vault since parser	encoded relative block delay matures	raw absolute integer is accepted as a delay
F1	Factory locality preserves untouched rights	proof bundle and rights-dependency graph	touched leaves prove affected rights only	shared reserve or exit right changes without authorisation
F2	Factory locality is not mint authority	local validity and conservation predicates	value right decreases or vault-delta-bound increases	Merkle-only proof increases ReserveClaim or Balance
F3	Factory vault materialisation matches signed descriptors	descriptor and delta materialisation checks	FactoryVault capacity, type, and amount match the signed descriptor	xUDT token delta is correct but the recreated FactoryVault capacity drifts
F4	Factory transition family is fixed before proof interpretation	bounded outer claim plus body commitment	admitted local proof is checked under its declared family	same proof body is reinterpreted as a weaker path

Table 5: Construction audit matrix for Morph Channel.